

Machine Learning Python Assignment

Reg No: 242BCA30

Name: Jonathan Marcus C

Pandas

P101_Load_Data.py

Program:

```
import pandas as pd

test = pd.DataFrame({
    "name":    ["Alice", "Bob", "Charlie", "Diana", "Eve"],
    "age":     [25, 30, 22, 35, 28],
    "column_name": [5, 15, 8, 20, 12],
    "category": ["A", "B", "A", "C", "B"],
    "date":    ["2024-01-15", "2024-03-22", "2024-06-05", "2024-09-10", "2024-11-30"]
})

test.to_csv("data.csv", index=False)

df = pd.read_csv("data.csv")
print("DataFrame loaded from data.csv:")
print(df)
```

Output:

```
DataFrame loaded from data.csv:
   name  age  column_name  category      date
0  Alice   25           5         A  2024-01-15
1   Bob   30          15         B  2024-03-22
2 Charlie  22           8         A  2024-06-05
3  Diana  35          20         C  2024-09-10
4   Eve   28          12         B  2024-11-30
```

P102_Display_Rows.py

Program:

```
import pandas as pd

df = pd.DataFrame({
    "name":    ["Alice", "Bob", "Charlie", "Diana", "Eve", "Frank", "Grace"],
    "age":     [25, 30, 22, 35, 28, 40, 19],
    "column_name": [5, 15, 8, 20, 12, 7, 18],
    "category": ["A", "B", "A", "C", "B", "C", "A"]
})

print("First 5 rows:")
print(df.head(5))
```

Output:

```
First 5 rows:
   name  age  column_name  category
0  Alice   25           5         A
1   Bob   30          15         B
2 Charlie   22           8         A
3  Diana   35          20         C
4   Eve   28          12         B
```

P103_No_Rows_Cols.py

Program:

```
import pandas as pd

df = pd.DataFrame({
    "name":    ["Alice", "Bob", "Charlie", "Diana", "Eve"],
    "age":     [25, 30, 22, 35, 28],
    "score":   [88, 75, 92, 60, 85],
    "category": ["A", "B", "A", "C", "B"]
})

rows, cols = df.shape
print("Number of rows:", rows)
print("Number of columns:", cols)
```

Output:

```
Number of rows: 5
Number of columns: 4
```

P104_Filter_Data.py

Program:

```
import pandas as pd

df = pd.DataFrame({
    "name":    ["Alice", "Bob", "Charlie", "Diana", "Eve"],
    "column_name": [5, 15, 8, 20, 12],
    "category": ["A", "B", "A", "C", "B"]
})

df_filtered = df[df["column_name"] > 10]
print("Original DataFrame:\n", df)
print("\nFiltered (column_name > 10):\n", df_filtered)
```

Output:

```
Original DataFrame:
   name  column_name category
0  Alice           5        A
1   Bob          15        B
2 Charlie           8        A
3  Diana          20        C
4   Eve          12        B

Filtered (column_name > 10):
   name  column_name category
1   Bob          15        B
3  Diana          20        C
4   Eve          12        B
```

P105_Missing_Data.py

Program:

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    "name": ["Alice", "Bob", None, "Diana", "Eve"],
    "age": [25, np.nan, 22, 35, np.nan],
    "score": [88, 75, np.nan, 60, 85]
})

print("Count:\n", df.isnull().sum())

df["age"] = df["age"].fillna(df["age"].mean())
df["score"] = df["score"].fillna(df["score"].mean())
df = df.dropna()
print("\nNew DataFrame:\n", df)
```

Output:

```
Count:
  name      1
  age      2
  score    1
dtype: int64
```

```
New DataFrame:
   name  age  score
0  Alice  25.000000  88.0
1   Bob   27.333333  75.0
3  Diana  35.000000  60.0
4   Eve   27.333333  85.0
```

P106_Group_Aggregate.py

Program:

```
import pandas as pd

df = pd.DataFrame({
    "category": ["A", "B", "A", "C", "B", "C", "A"],
    "value":    [10, 20, 30, 40, 50, 60, 70]
})

gmean = df.groupby("category")["value"].mean()

print("DataFrame:\n", df)
print("\nMean values in each category:\n", gmean)
```

Output:

```
DataFrame:
   category  value
0         A     10
1         B     20
2         A     30
3         C     40
4         B     50
5         C     60
6         A     70

Mean values in each category:
category
A      36.666667
B      35.000000
C      50.000000
Name: value, dtype: float64
```

P107_Merge_DataFrames.py

Program:

```
import pandas as pd

df1 = pd.DataFrame({
    "id": [1, 2, 3, 4],
    "name": ["Alice", "Bob", "Charlie", "Diana"]
})

df2 = pd.DataFrame({
    "id": [2, 3, 4, 5],
    "score": [85, 90, 78, 92]
})

merged = pd.merge(df1, df2, on="id")
print("DF1:\n", df1)
print("\nDF2:\n", df2)
print("\nMerged DF:\n", merged)
```

Output:

DF1:

	id	name
0	1	Alice
1	2	Bob
2	3	Charlie
3	4	Diana

DF2:

	id	score
0	2	85
1	3	90
2	4	78
3	5	92

Merged DF:

	id	name	score
0	2	Bob	85
1	3	Charlie	90
2	4	Diana	78

P108_Bar_Chart.py

Program:

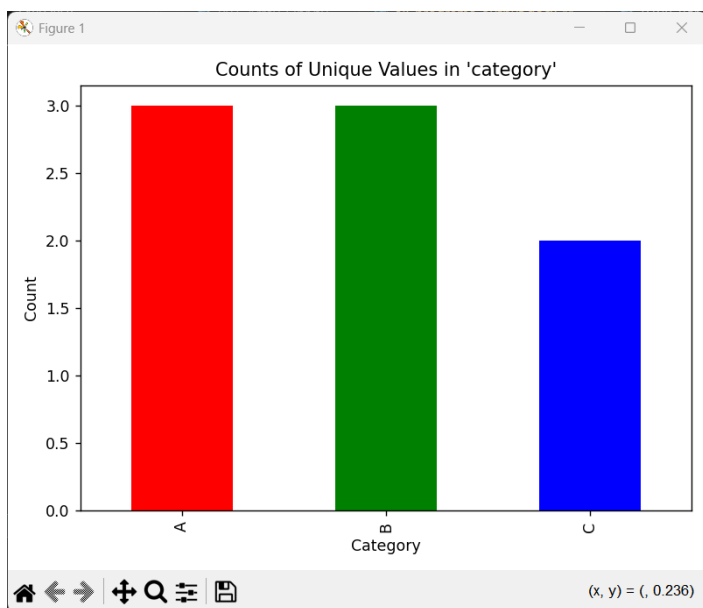
```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.DataFrame({
    "category": ["A", "B", "A", "C", "B", "C", "A", "B"]
})

values = df["category"].value_counts()
values.plot(kind="bar", color=["red", "green", "blue"])

plt.title("Counts of Unique Values in 'category'")
plt.xlabel("Category")
plt.ylabel("Count")
plt.tight_layout()
plt.show()
```

Output:



P109_Datetime_Ops.py

Program:

```
import pandas as pd

df = pd.DataFrame({
    "event": ["A", "B", "C", "D"],
    "date": ["2024-01-15", "2024-03-22", "2024-06-05", "2024-11-30"]
})

df["date_datetime"] = pd.to_datetime(df["date"], format="%Y-%m-%d")
df["month"] = df["date"].dt.month
print(df)
```

Output:

```
[stderr]
Traceback (most recent call last):
  File "/mnt/documents/Essentials/SJU/SJU Programming/S5 - ML/Pandas/P109_Datetime_Ops.py", line 9, in <module>
    df["month"] = df["date"].dt.month
  File "/home/sujan/.local/lib/python3.10/site-packages/pandas/core/generic.py", line 6321, in __getattr__
    return object.__getattribute__(self, name)
  File "/home/sujan/.local/lib/python3.10/site-packages/pandas/core/accessor.py", line 224, in __get__
    accessor_obj = self._accessor(obj)
  File "/home/sujan/.local/lib/python3.10/site-packages/pandas/core/indexes/accessors.py", line 643, in __new__
    raise AttributeError("Can only use .dt accessor with datetimelike values")
AttributeError: Can only use .dt accessor with datetimelike values. Did you mean: 'at'?
```

P110_Save_Data.py

Program:

```
import pandas as pd

df = pd.DataFrame({
    "name":  ["Alice", "Bob", "Charlie"],
    "score": [88, 75, 92],
    "category": ["A", "B", "A"]
})

df["grade"] = df["score"].apply(lambda x: "Pass" if x >= 80 else "Fail")

df.to_csv("output_data.csv", index=False)
print("\nOutput CSV:")
with open("output_data.csv", "r") as file:
    print(file.read())
```

Output:

```
Output CSV:
name,score,category,grade
Alice,88,A,Pass
Bob,75,B,Fail
Charlie,92,A,Pass
```

P201_Load_Iris.py

Program:

```
import pandas as pd
from sklearn.datasets import load_iris

iris = load_iris(as_frame=True)
df = iris.frame
df["species"] = df["target"].map({0: "setosa", 1: "versicolor", 2: "virginica"})

print("\nFirst 5 rows:")
print(df.head())
```

Output:

```
First 5 rows:
   sepal length (cm)  sepal width (cm)  ...  target  species
0                5.1                3.5  ...      0    setosa
1                4.9                3.0  ...      0    setosa
2                4.7                3.2  ...      0    setosa
3                4.6                3.1  ...      0    setosa
4                5.0                3.6  ...      0    setosa

[5 rows x 6 columns]
```

P202_Filter_Iris.py

Program:

```
import pandas as pd
from sklearn.datasets import load_iris

iris = load_iris(as_frame=True)
df = iris.frame
df["species"] = df["target"].map({0: "setosa", 1: "versicolor", 2: "virginica"})

df_filtered = df[(df["sepal length (cm)"] > 5) & (df["species"] == "setosa")]

print("Rows where sepal_length > 5 and species == setosa:")
print(df_filtered)
```

Output:

```
Rows where sepal_length > 5 and species == setosa:
   sepal length (cm)  sepal width (cm)  ...  target  species
0                5.1                3.5  ...      0   setosa
5                5.4                3.9  ...      0   setosa
10               5.4                3.7  ...      0   setosa
14               5.8                4.0  ...      0   setosa
15               5.7                4.4  ...      0   setosa
16               5.4                3.9  ...      0   setosa
17               5.1                3.5  ...      0   setosa
18               5.7                3.8  ...      0   setosa
19               5.1                3.8  ...      0   setosa
20               5.4                3.4  ...      0   setosa
21               5.1                3.7  ...      0   setosa
23               5.1                3.3  ...      0   setosa
27               5.2                3.5  ...      0   setosa
28               5.2                3.4  ...      0   setosa
31               5.4                3.4  ...      0   setosa
32               5.2                4.1  ...      0   setosa
33               5.5                4.2  ...      0   setosa
36               5.5                3.5  ...      0   setosa
39               5.1                3.4  ...      0   setosa
44               5.1                3.8  ...      0   setosa
46               5.1                3.8  ...      0   setosa
48               5.3                3.7  ...      0   setosa

[22 rows x 6 columns]
```

P203_Handle_Missing.py

Program:

```
import pandas as pd
import numpy as np
from sklearn.datasets import load_iris

iris = load_iris(as_frame=True)
df = iris.frame

df.iloc[0, 0] = np.nan
df.iloc[5, 2] = np.nan

print("Missing values before:\n", df.isnull().sum())
df.fillna(df.mean(numeric_only=True), inplace=True)
print("\nMissing values after:\n", df.isnull().sum())
```

Output:

```
Missing values before:
  sepal length (cm)    1
  sepal width (cm)     0
  petal length (cm)    1
  petal width (cm)     0
  target              0
dtype: int64
```

```
Missing values after:
  sepal length (cm)    0
  sepal width (cm)     0
  petal length (cm)    0
  petal width (cm)     0
  target              0
dtype: int64
```

P204_Group_Stats.py

Program:

```
import pandas as pd
from sklearn.datasets import load_iris

iris = load_iris(as_frame=True)
df = iris.frame
df["species"] = df["target"].map({0: "setosa", 1: "versicolor", 2: "virginica"})

numeric_cols = ["sepal length (cm)", "sepal width (cm)", "petal length (cm)", "petal width (cm)"]
stats = df.groupby("species")[numeric_cols].agg(["mean", "median", "std"])

print(stats)
```

Output:

```
      sepal length (cm)      ... petal width (cm)
      mean median      std  ...      mean median
std
species
setosa      5.006      5.0  0.352490  ...      0.246      0.2
0.105386
versicolor      5.936      5.9  0.516171  ...      1.326      1.3
0.197753
virginica      6.588      6.5  0.635880  ...      2.026      2.0
0.274650

[3 rows x 12 columns]
```

P205_Merge_Students.py

Program:

```
import pandas as pd

students = pd.DataFrame({
    "student_id": [101, 102, 103, 104],
    "name":      ["Rahul", "Brian", "Yash", "Vigesh"]
})

courses = pd.DataFrame({
    "student_id": [101, 102, 103, 105],
    "course":     ["Math", "Science", "English", "History"],
    "grade":      ["A", "B+", "A-", "B"]
})

merged = pd.merge(students, courses, on="student_id", how="inner")
print("Students:\n", students)
print("\nCourses:\n", courses)
print("\nMerged:\n", merged)
```

Output:

```
Students:
   student_id  name
0          101  Rahul
1          102  Brian
2          103   Yash
3          104  Vigesh

Courses:
   student_id  course grade
0          101    Math    A
1          102  Science  B+
2          103  English  A-
3          105  History    B

Merged:
   student_id  name  course grade
0          101  Rahul    Math    A
1          102  Brian  Science  B+
2          103   Yash  English  A-
```

P206_Time_Series.py

Program:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

dates = pd.date_range(start="2026-01-01", periods=30, freq="B")
prices = 100 + np.cumsum(np.random.randn(30))

df = pd.DataFrame({"date": dates, "price": prices})
df.set_index("date", inplace=True)

df["daily_return"] = df["price"].pct_change()
print(df.head(10))

fig, (price_plot, dr_plot) = plt.subplots(2, 1, figsize=(8, 5))
price_plot.plot(df["price"], label="Price")
price_plot.set_title("Stock Price")
dr_plot.bar(df.index, df["daily_return"], label="Daily Return", color="red")
dr_plot.set_title("Daily Returns")
plt.tight_layout()
plt.show()
```

Output:

	price	daily_return
date		
2026-01-01	98.486668	NaN
2026-01-02	98.635305	0.001509
2026-01-05	97.039186	-0.016182
2026-01-06	98.621567	0.016307
2026-01-07	99.620498	0.010129
2026-01-08	99.326530	-0.002951
2026-01-09	100.939639	0.016240
2026-01-12	100.215729	-0.007172
2026-01-13	100.285994	0.000701
2026-01-14	100.769094	0.004817

P207_Pivot_Table.py

Program:

```
import pandas as pd

df = pd.DataFrame({
    "month":  ["Jan", "Jan", "Feb", "Feb", "Mar", "Mar"],
    "category": ["Electronics", "Clothing", "Electronics", "Clothing", "Electronics", "Clothing"],
    "sales":  [5000, 2000, 4500, 2500, 6000, 3000]
})

pivot = pd.pivot_table(df, values="sales", index="category", columns="month", aggfunc="sum")

print("Pivot Table:")
print(pivot)
```

Output:

```
Pivot Table:
month      Feb   Jan   Mar
category
Clothing    2500  2000  3000
Electronics 4500  5000  6000
```

P208_Visualizations.py

Program:

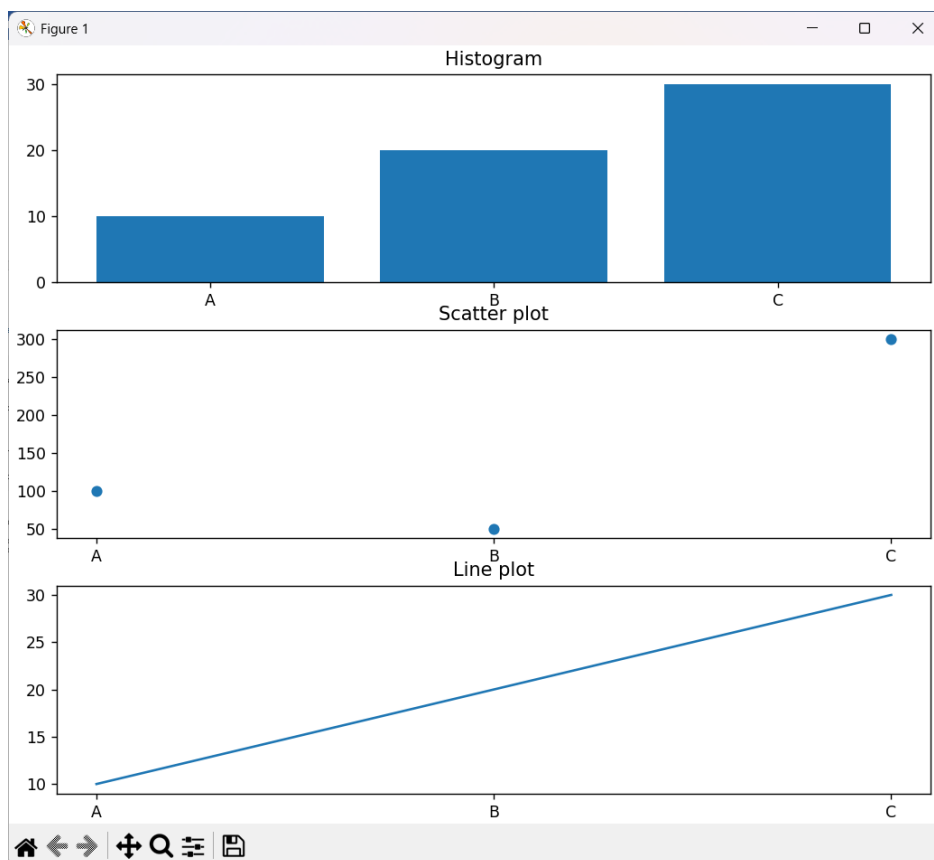
```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.DataFrame({
    "name": ["A","B","C"],
    "age": [10,20,30],
    "score": [100,50,300]
})

fig, [hist, scat, line] = plt.subplots(3,1, figsize=(10,10))

hist.bar(df["name"], df["age"], label="Histogram")
hist.set_title("Histogram")
scat.scatter(df["name"], df["score"], label="Scatter plot")
scat.set_title("Scatter plot")
line.plot(df["name"], df["age"], label="Line plot")
line.set_title("Line plot")
plt.tight_layout()
plt.show()
```

Output:



P209_Data_Transform.py

Program:

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    "Var": ["A","B","C","D"],
    "Val": [x**2 for x in range(1,5)]
})

df["ValSq"] = df["Val"].apply(np.sqrt)
print(df)
```

Output:

	Var	Val	ValSq
0	A	1	1.0
1	B	4	2.0
2	C	9	3.0
3	D	16	4.0

P210_One_Hot_Encoding.py

Program:

```
import pandas as pd

df = pd.DataFrame({
    "name": ["Rahul", "Sam", "Raj", "Tom"],
    "department": ["HR", "IT", "HR", "Sales"]
})

print("Before Encoding:")
print(df)

df_encoded = pd.get_dummies(df, columns=["department"])

print("\nAfter Encoding:")
print(df_encoded)
```

Output:

```
Before Encoding:
   name department
0  Rahul         HR
1   Sam         IT
2   Raj         HR
3   Tom        Sales

After Encoding:
   name  department_HR  department_IT  department_Sales
0  Rahul             True             False             False
1   Sam             False              True             False
2   Raj             True             False             False
3   Tom             False             False              True
```

NumPy

P01_Array_1D.py

Program:

```
import numpy as np

nums = np.arange(10)
print("1D Array:", nums)
```

Output:

```
1D Array: [0 1 2 3 4 5 6 7 8 9]
```

P02_Array_2D.py

Program:

```
import numpy as np

nums = np.random.rand(3, 4)
print("2D Array (3x4):\n", nums)
```

Output:

```
2D Array (3x4) :
[[0.59852957 0.08503716 0.86538912 0.06708755]
 [0.49057565 0.53566855 0.98722912 0.60731581]
 [0.9768445  0.82851397 0.04867291 0.96759762]]
```

P03_Mean_Std.py

Program:

```
import numpy as np

nums = np.array([4, 7, 2, 9, 1, 5, 8, 3, 6])
print("Array:", nums)
print("Mean:", np.mean(nums))
print("Std Dev:", np.std(nums))
```

Output:

```
Array: [4 7 2 9 1 5 8 3 6]
Mean: 5.0
Std Dev: 2.581988897471611
```

P04_Normalize.py

Program:

```
import numpy as np

# Normalize: (subtract the mean and divide by std deviation)z
nums = np.array([10.0, 20.0, 30.0, 40.0, 50.0])
normalized = (nums - np.mean(nums)) / np.std(nums)
print("Original:", nums)
print("Normalized:", normalized)
```

Output:

```
Original: [10. 20. 30. 40. 50.]
Normalized: [-1.41421356 -0.70710678  0.          0.70710678  1.41421356]
```

P05_Slice_Column.py

Program:

```
import numpy as np

nums = np.array([[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]])
tc = nums[:, 2]
print("2D Array:\n", nums)
print("Third Column:", tc)
```

Output:

```
2D Array:
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]
Third Column: [ 3  7 11]
```

P06_Reverse_Array.py

Program:

```
import numpy as np

nums = np.array([1, 2, 3, 4, 5, 6, 7])
rnums = nums[::-1]
print("Original:", nums)
print("Reversed:", rnums)
```

Output:

```
Original: [1 2 3 4 5 6 7]
Reversed: [7 6 5 4 3 2 1]
```

P07_Matrix_Multiply.py

Program:

```
import numpy as np

A = np.array([[1, 2, 3],
              [4, 5, 6]])

B = np.array([[7, 8, 9, 10],
              [11, 12, 13, 14],
              [15, 16, 17, 18]])

r = np.matmul(A, B)
print("A:\n", A)
print("B:\n", B)
print("A * B:\n", r)
```

Output:

```
A:
[[ 1  2  3]
 [ 4  5  6]]
B:
[[ 7  8  9 10]
 [11 12 13 14]
 [15 16 17 18]]
A * B:
[[ 74  80  86  92]
 [173 188 203 218]]
```

P08_Determinant.py

Program:

```
import numpy as np

m = np.array([[1, 2, 5],
              [4, 7, 3],
              [4, 2, 1]])

det = np.linalg.det(m)
print("Matrix:\n", m)
print("Determinant:", det)
```

Output:

```
Matrix:
[[ 1  2  5]
 [ 4  7  3]
 [ 4  2  1]]
Determinant: -83.00000000000003
```

P09_Linspace.py

Program:

```
import numpy as np

arr = np.arange(0, 10, 1)
print("Evenly spaced values:", arr)
```

Output:

```
Evenly spaced values: [0 1 2 3 4 5 6 7 8 9]
```

P10_Identity_Matrix.py

Program:

```
import numpy as np

identity = np.eye(3)
print("3x3 Identity Matrix:\n", identity)
```

Output:

```
3x3 Identity Matrix:
[[1.  0.  0.]
 [0.  1.  0.]
 [0.  0.  1.]]
```

P11_Reshape.py

Program:

```
import numpy as np

nums = np.arange(10)
nnums = nums.reshape(2, 5)
print("1D Array:", nums)
print("Reshaped (2x5):\n", nnums)
```

Output:

```
1D Array: [0 1 2 3 4 5 6 7 8 9]
Reshaped (2x5):
[[0 1 2 3 4]
 [5 6 7 8 9]]
```

P12_Stack_Arrays.py

Program:

```
import numpy as np

a = np.array([[1, 2, 3],
              [4, 5, 6]])

b = np.array([[7, 8, 9],
              [10, 11, 12]])

v = np.vstack((a, b))
h = np.hstack((a, b))
print("Array A:\n", a)
print("Array B:\n", b)
print("Vertical Stack:\n", v)
print("Horizontal Stack:\n", h)
```

Output:

```
Array A:
[[1 2 3]
 [4 5 6]]
Array B:
[[ 7  8  9]
 [10 11 12]]
Vertical Stack:
[[ 1  2  3]
 [ 4  5  6]
 [ 7  8  9]
 [10 11 12]]
Horizontal Stack:
[[ 1  2  3  7  8  9]
 [ 4  5  6 10 11 12]]
```

P13_Boolean_Extract.py

Program:

```
import numpy as np

nums = np.random.rand(10)
filtered = nums[nums > 0.5]
print("Random Array:", nums)
print("Elements > 0.5:", filtered)
```

Output:

```
Random Array: [0.66442194 0.54132514 0.28927237 0.82833302 0.40707602 0.529742
 0.39158624 0.67119353 0.85264667 0.24947224]
Elements > 0.5: [0.66442194 0.54132514 0.82833302 0.529742    0.67119353
 0.85264667]
```

P14_Replace_Negatives.py

Program:

```
import numpy as np

nums = np.array([3, -1, 4, -1, 5, -9, 2, 6, -5])
nums[nums < 0] = 0
print("New Array: ", nums)
```

Output:

```
New Array:  [3 0 4 0 5 0 2 6 0]
```

P15_Axis_Mean.py

Program:

```
import numpy as np

nums = np.random.randint(1, 20, size=(3, 4))
print("2D Array:\n", nums)
print("Mean along axis 0 (column):", np.mean(nums, axis=0))
print("Mean along axis 1 (row):", np.mean(nums, axis=1))
```

Output:

```
2D Array:
[[ 6  8  8  4]
 [15  8  9 17]
 [14 16  6  3]]
Mean along axis 0 (column): [11.66666667 10.66666667  7.66666667  8.        ]
Mean along axis 1 (row): [ 6.5  12.25  9.75]
```

P16_Min_Max.py

Program:

```
import numpy as np

nums = np.array([15, 3, 42, 7, 28, 1, 99, 56])
print("Array:", nums)
print("Minimum:", np.min(nums))
print("Maximum:", np.max(nums))
```

Output:

```
Array: [15  3 42  7 28  1 99 56]
Minimum: 1
Maximum: 99
```

P17_Broadcast_Add.py

Program:

```
import numpy as np

nums = np.array([1, 2, 3, 4, 5])
cv = 10
r = nums + cv
print("Original:", nums)
print(f"After adding {cv}:", r)
```

Output:

```
Original: [1 2 3 4 5]
After adding 10: [11 12 13 14 15]
```

P18_Broadcast_Multiply.py

Program:

```
import numpy as np

arr = np.array([[1, 2, 3],
                [4, 5, 6],
                [7, 8, 9]])

cvs = np.array([2, 3, 4])
result = arr * cvs[:, np.newaxis]
print("Original:\n", arr)
print("Constants:", cvs)
print("Result:\n", result)
```

Output:

```
Original:
[[1 2 3]
 [4 5 6]
 [7 8 9]]
Constants: [2 3 4]
Result:
[[ 2  4  6]
 [12 15 18]
 [28 32 36]]
```

P19_Custom_Function.py

Program:

```
import numpy as np

def power(x):
    return x ** 2

v = np.vectorize(power)
nums = np.array([0, 1, 2, 3, 4, 5])
r = v(nums)
print("Input:", nums)
print("f(x) = x^2 ->\n", r)
```

Output:

```
Input: [0 1 2 3 4 5]
f(x) = x^2 ->
[ 0  1  4  9 16 25]
```

P20_Linear_Equations.py

Program:

```
import numpy as np

A = np.array([[2, 1], [1, 3]])

b = np.array([5, 10])

r = np.linalg.solve(A, b)

print("Result: ",r)
```

Output:

```
Result: [1. 3.]
```

Matplotlib

P01_Line_Plot.py

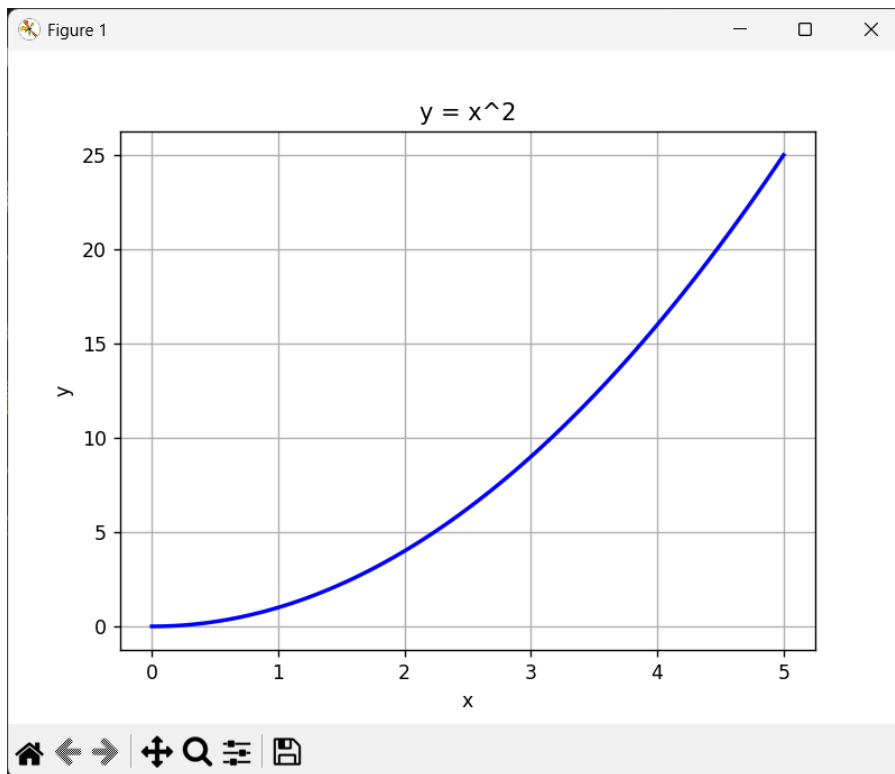
Program:

```
import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(0, 5, 100)
y = x ** 2

plt.plot(x, y, color="blue", linewidth=2)
plt.title("y = x^2")
plt.xlabel("x")
plt.ylabel("y")
plt.grid(True)
plt.show()
```

Output:



P02_Scatter_Plot.py

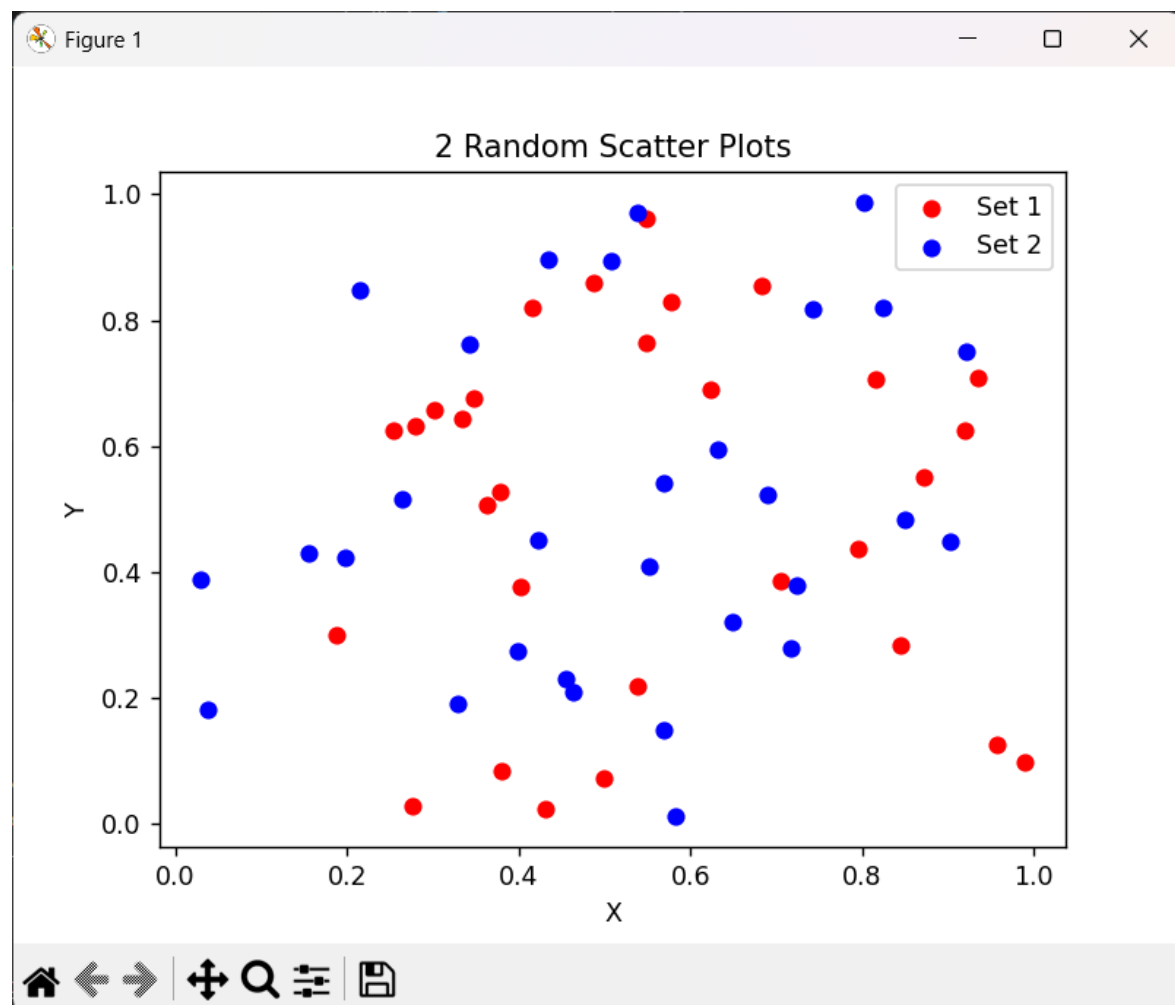
Program:

```
import matplotlib.pyplot as plt
import numpy as np

X1, Y1 = np.random.rand(30), np.random.rand(30)
X2, Y2 = np.random.rand(30), np.random.rand(30)

plt.scatter(X1, Y1, color="red", label="Set 1")
plt.scatter(X2, Y2, color="blue", label="Set 2")
plt.title("2 Random Scatter Plots")
plt.xlabel("X")
plt.ylabel("Y")
plt.legend()
plt.show()
```

Output:



P03_Histogram.py

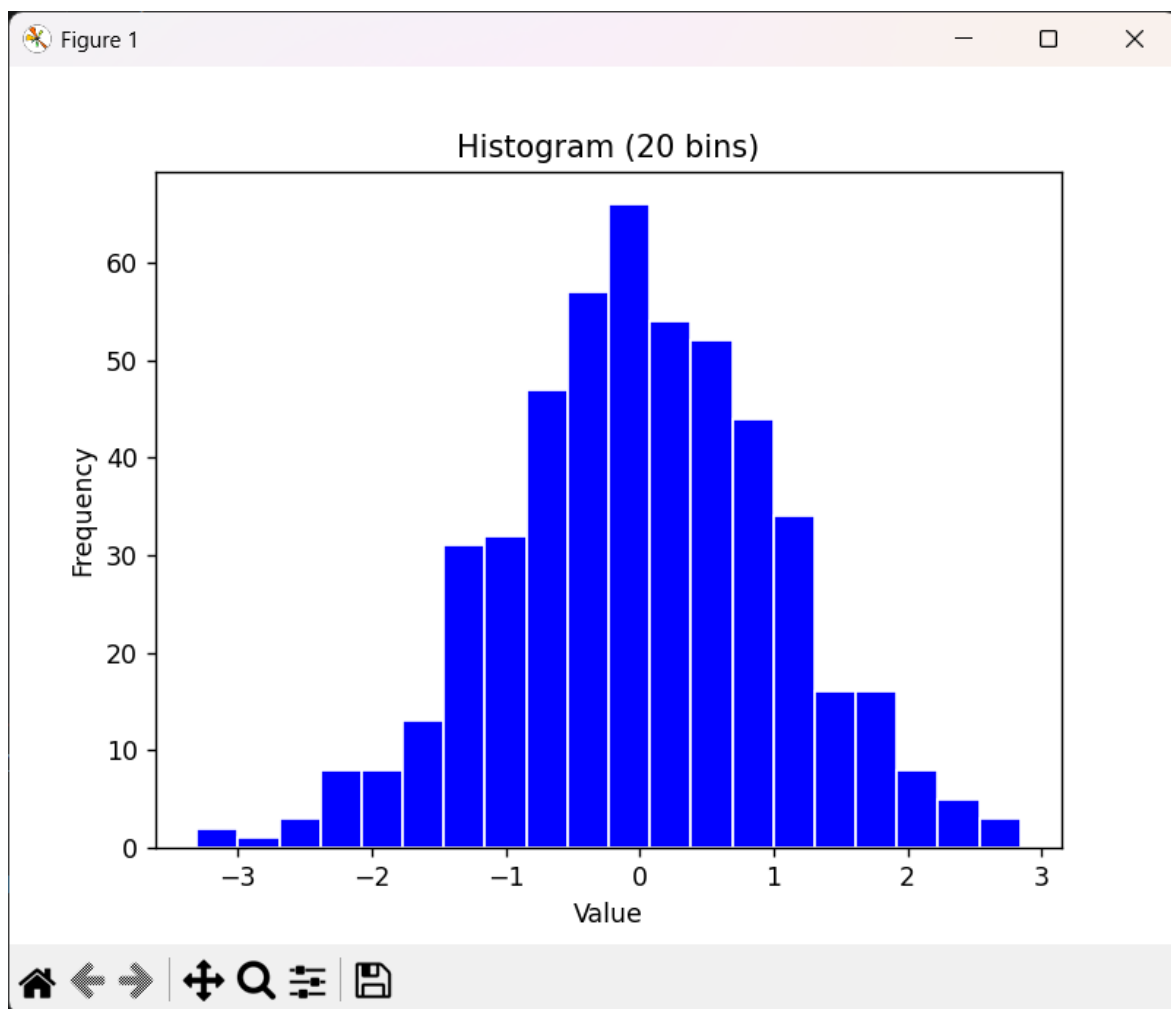
Program:

```
import matplotlib.pyplot as plt
import numpy as np

data = np.random.randn(500)
print(data)

plt.hist(data, bins=20, color="blue", edgecolor="white")
plt.title("Histogram (20 bins)")
plt.xlabel("Value")
plt.ylabel("Frequency")
plt.show()
```

Output:



P04_Bar_Chart.py

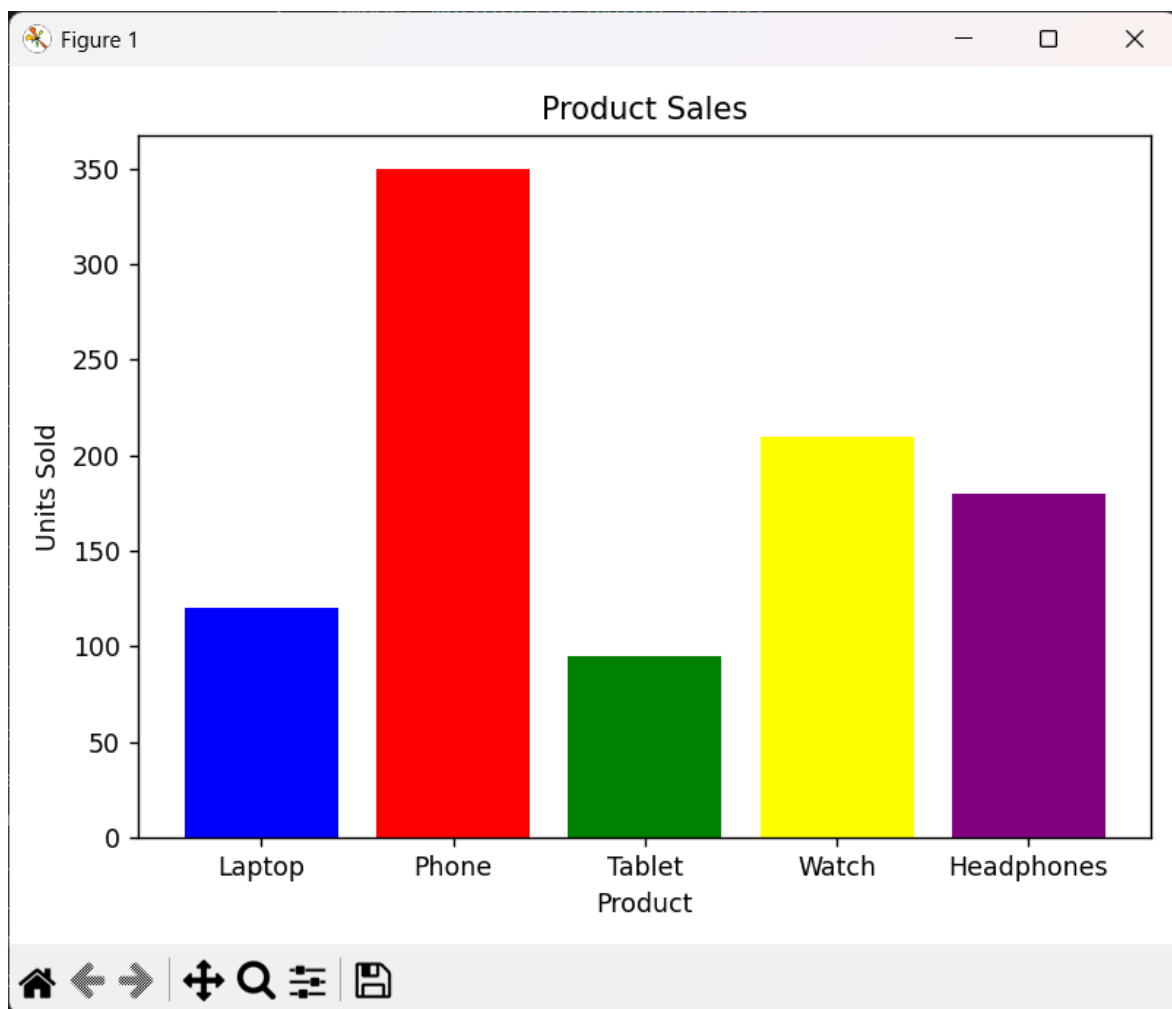
Program:

```
import matplotlib.pyplot as plt

products = ["Laptop", "Phone", "Tablet", "Watch", "Headphones"]
sales = [120, 350, 95, 210, 180]

plt.bar(products, sales, color=["blue", "red", "green", "yellow", "purple"])
plt.title("Product Sales")
plt.xlabel("Product")
plt.ylabel("Units Sold")
plt.tight_layout()
plt.show()
```

Output:



P05_Multiple_Subplots.py

Program:

```
import matplotlib.pyplot as plt
import numpy as np

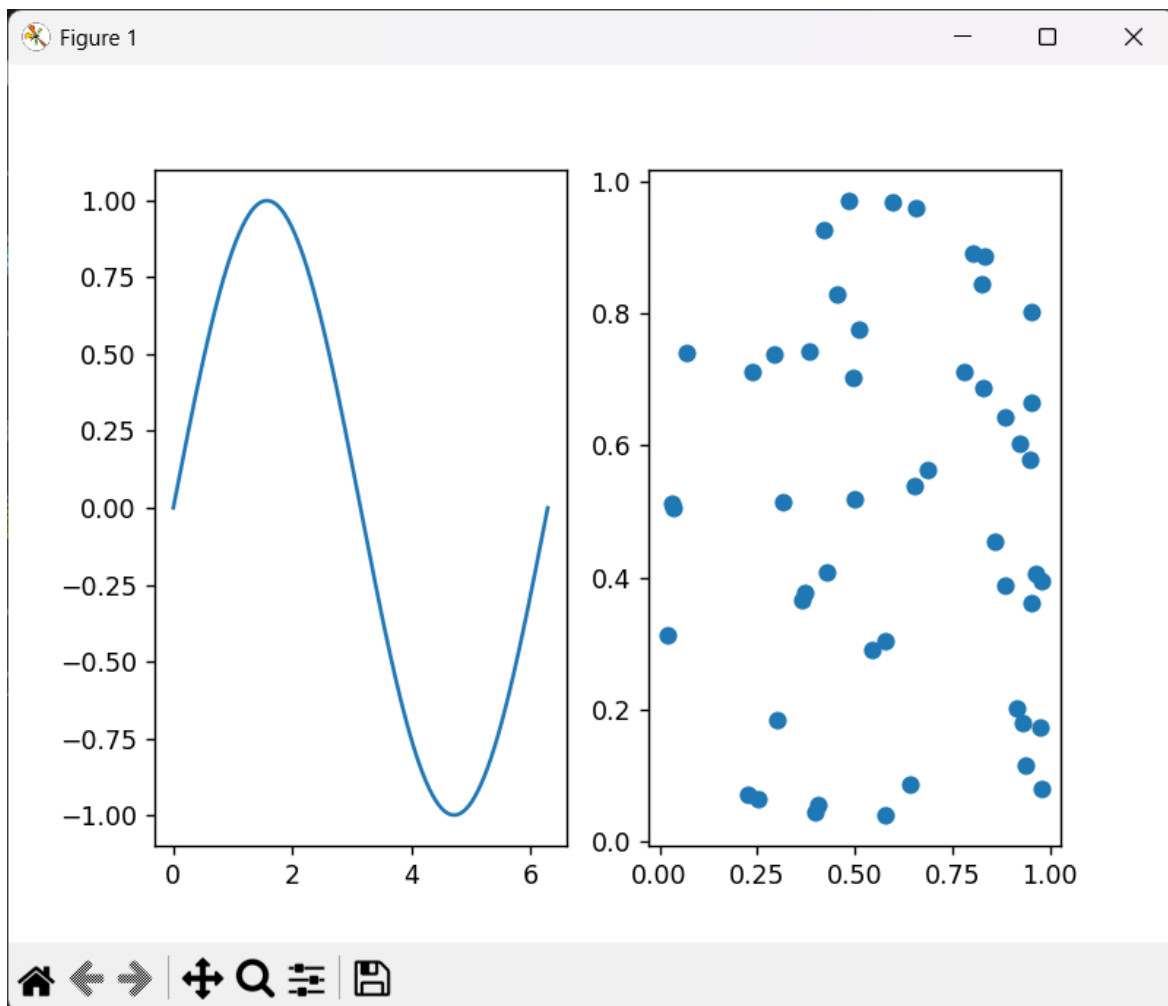
x = np.linspace(0, 2*np.pi, 200)

fig, (ax1, ax2) = plt.subplots(1, 2)

ax1.plot(x, np.sin(x))
ax2.scatter(np.random.rand(50), np.random.rand(50))

plt.show()
```

Output:



P06_Customize_Plot.py

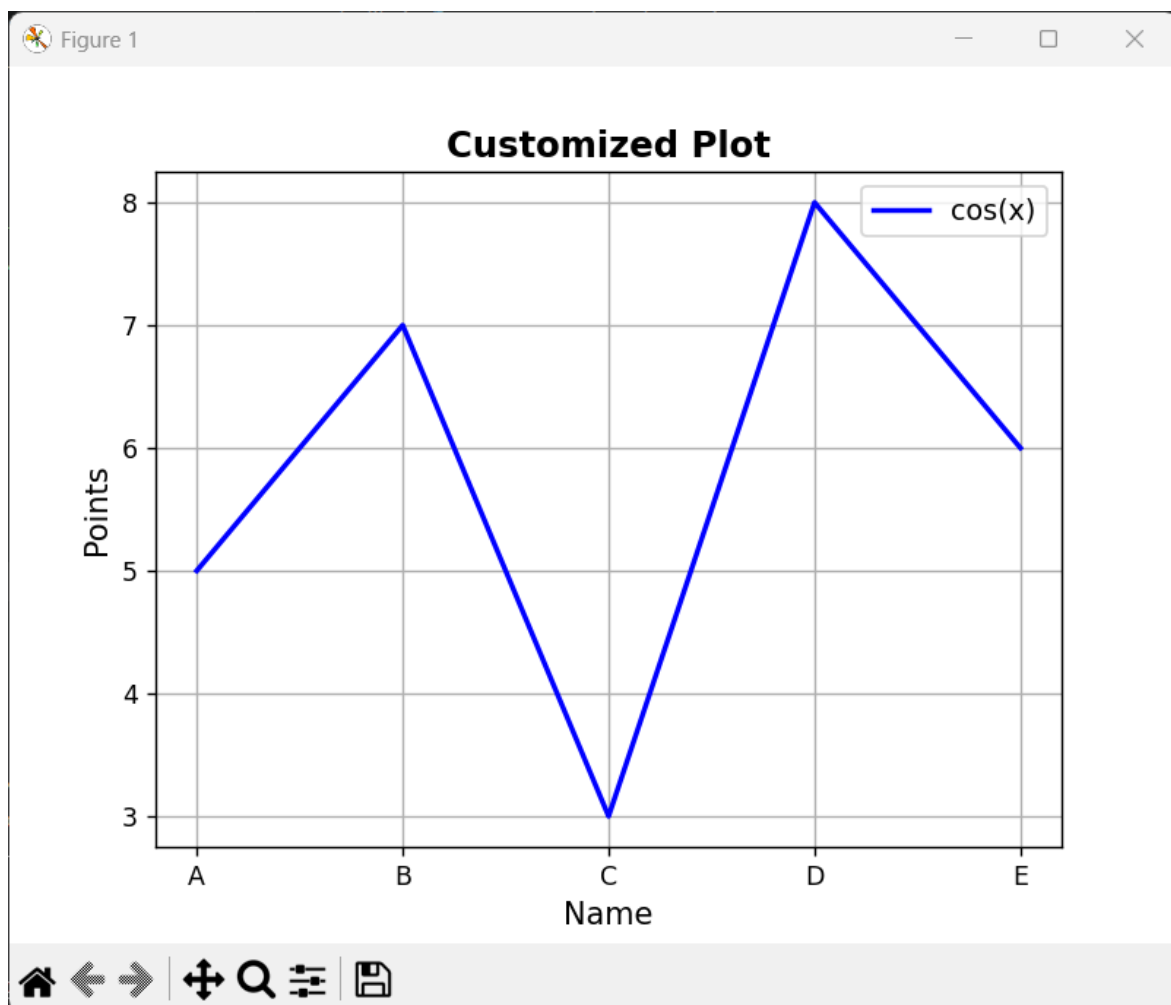
Program:

```
import matplotlib.pyplot as plt
import numpy as np

x = np.array(["A", "B", "C", "D", "E"])
y = np.array([5, 7, 3, 8, 6])

plt.plot(x, y, color="blue", linewidth=2, label="cos(x)")
plt.title("Customized Plot", fontsize=14, fontweight="bold")
plt.xlabel("Name", fontsize=12)
plt.ylabel("Points", fontsize=12)
plt.legend(fontsize=11)
plt.grid(True)
plt.show()
```

Output:



P07_Box_Plot.py

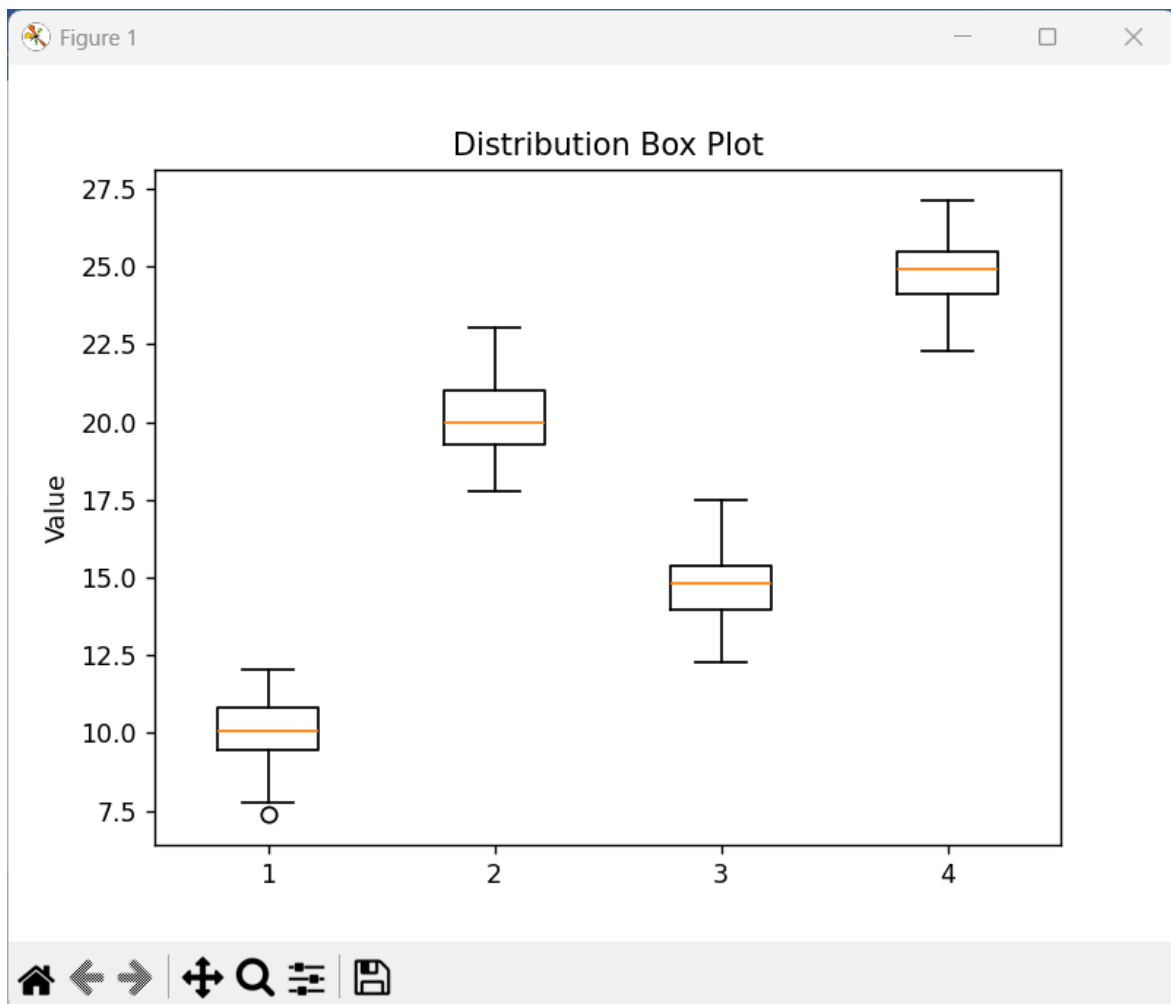
Program:

```
import matplotlib.pyplot as plt
import numpy as np

data = [np.random.normal(10, 1, 50),
        np.random.normal(20, 1, 50),
        np.random.normal(15, 1, 50),
        np.random.normal(25, 1, 50)]

plt.boxplot(data)
plt.title("Distribution Box Plot")
plt.ylabel("Value")
plt.show()
```

Output:



P08_Pie_Chart.py

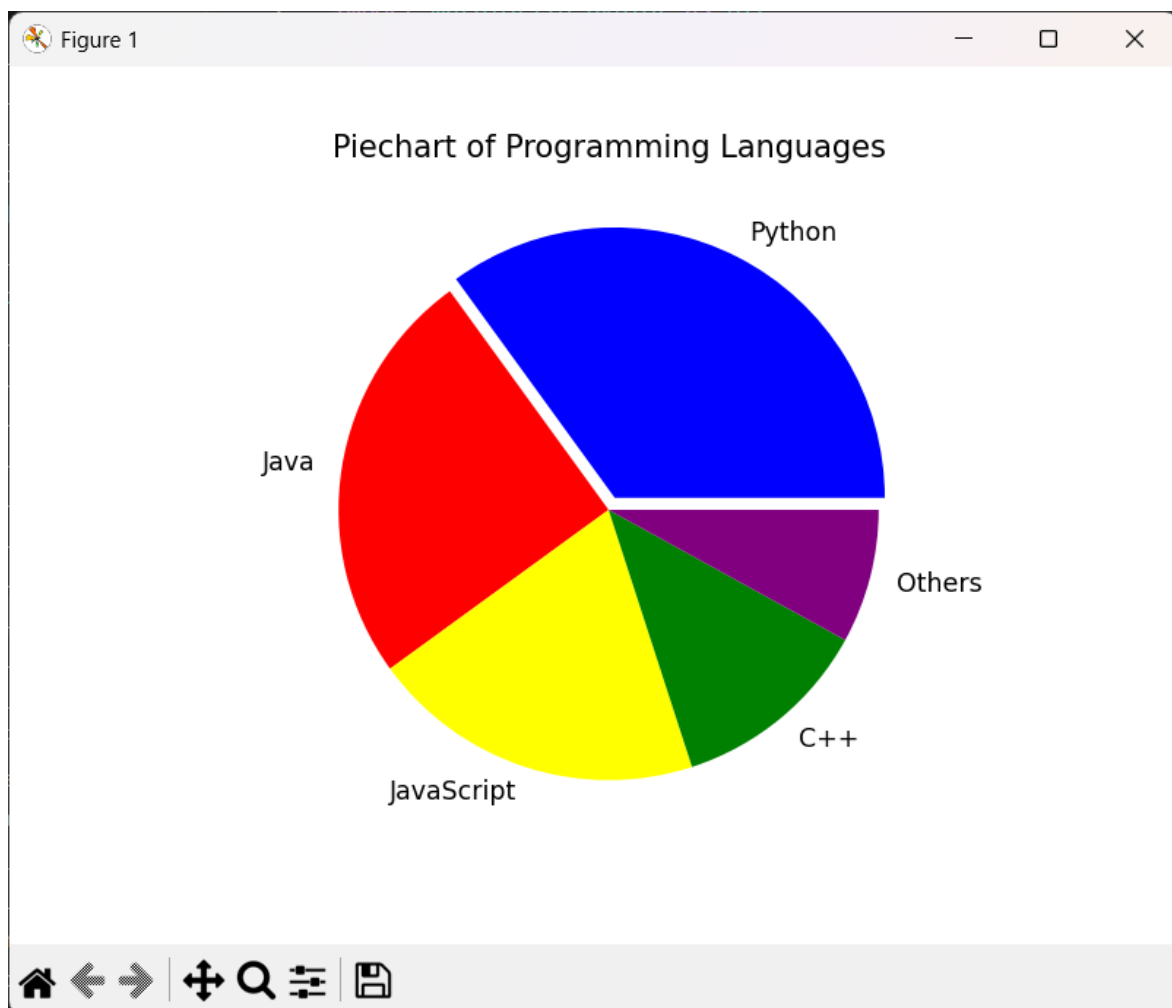
Program:

```
import matplotlib.pyplot as plt

labels = ["Python", "Java", "JavaScript", "C++", "Others"]
sizes = [35, 25, 20, 12, 8]
colors = ["blue", "red", "yellow", "green", "purple"]
explode = (0.05, 0, 0, 0, 0)

plt.pie(sizes, labels=labels, colors=colors, explode=explode)
plt.title("Piechart of Programming Languages")
plt.show()
```

Output:



P09_3D_Surface.py

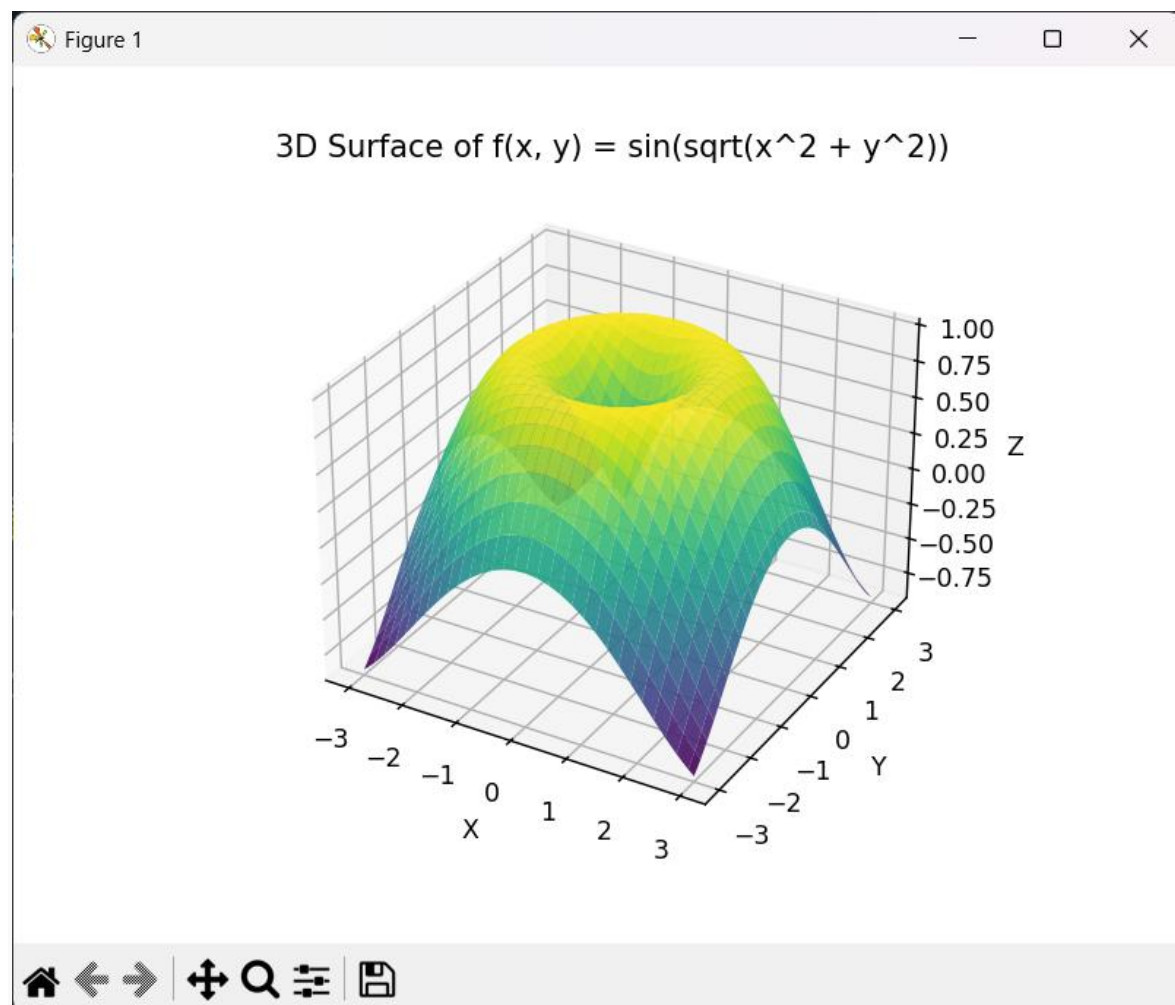
Program:

```
import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(-3, 3, 60)
y = np.linspace(-3, 3, 60)
X, Y = np.meshgrid(x, y)
Z = np.sin(np.sqrt(X**2 + Y**2))

fig = plt.figure()
ax = fig.add_subplot(111, projection="3d")
ax.plot_surface(X, Y, Z, cmap="viridis", alpha=0.9)
ax.set_title("3D Surface of  $f(x, y) = \sin(\sqrt{x^2 + y^2})$ ")
ax.set_xlabel("X")
ax.set_ylabel("Y")
ax.set_zlabel("Z")
plt.show()
```

Output:



P10_Error_Bars.py

Program:

```
import matplotlib.pyplot as plt
import numpy as np

x = np.arange(1, 8)
means = [np.random.normal(i*2, 1, 20).mean() for i in x]
stds = [np.random.normal(i*2, 1, 20).std() for i in x]

plt.errorbar(x, means, yerr=stds, fmt="-o", color="blue",
             ecol="red", elinewidth=2, capsize=5, label="Mean ± Std")
plt.title("Line Plot with Error Bars")
plt.xlabel("Data Point")
plt.ylabel("Value")
plt.legend()
plt.show()
```

Output:

